

FACULTATEA: Automatică și Calculatoare
SPECIALIZAREA: Calculatoare și Tehnologia Informației
DISCIPLINA: Proiectarea sistemelor numerice
PROIECT: CEAS CU RADIO PROGRAMABIL

Îndrumător laborator **Realizator**

Ing. Călugăr Gabriel-Cătălin **Florean Emilia**

Cuprins

1. Specificație proiect.....3

2. Schemă bloc, schemă bloc detaliata 3

3. Componente.....3

4. Cod VHDL.....3

5. Utilizare și semnificația notațiilor.....3

6. Justificarea soluției alese.....3

7. Rezultate.....3

8. Oportunități de dezvoltare.....3

1. Specificația proiectului

Să se proiecteze un automat pentru un ceas cu radio, programabil. Ceasul poate fi resetat, fixat la o oră dorită, pornit, oprit. Ceasul poate fi programat să pornească şi să oprească o sonerie sau să pornească şi să oprească radioul.

2. Schemă bloc, schemă detaliată

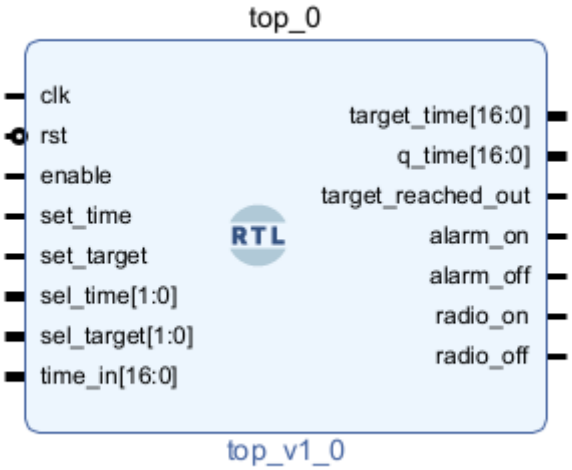


Fig. 1 – Schema Block simplă – Black Box

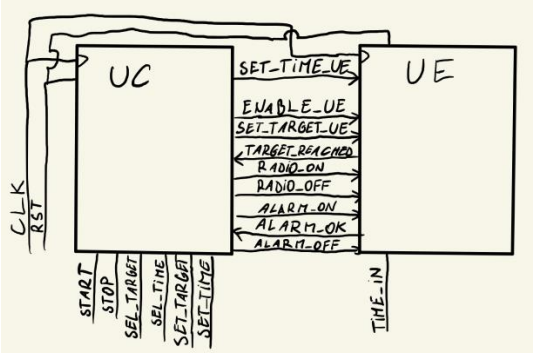


Fig. 2- UE si UC

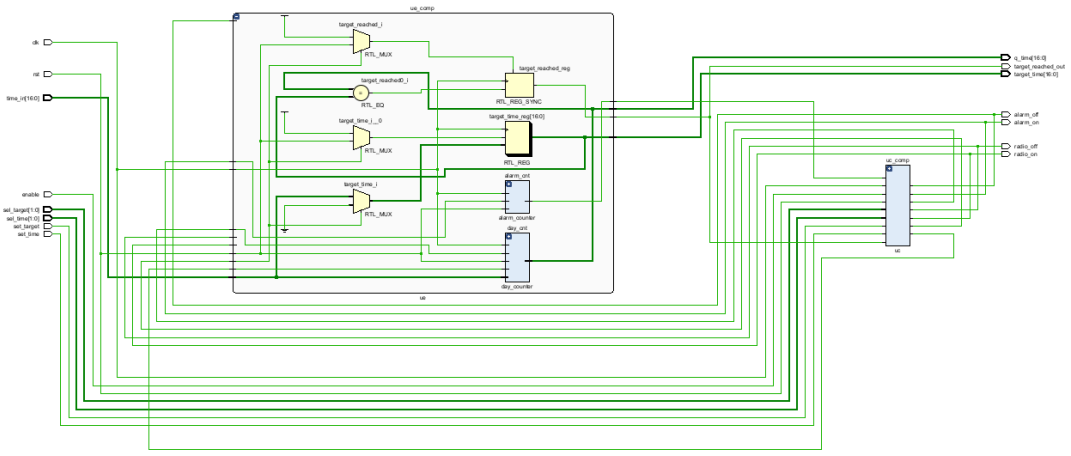


Fig. 3 – Schema detaliată

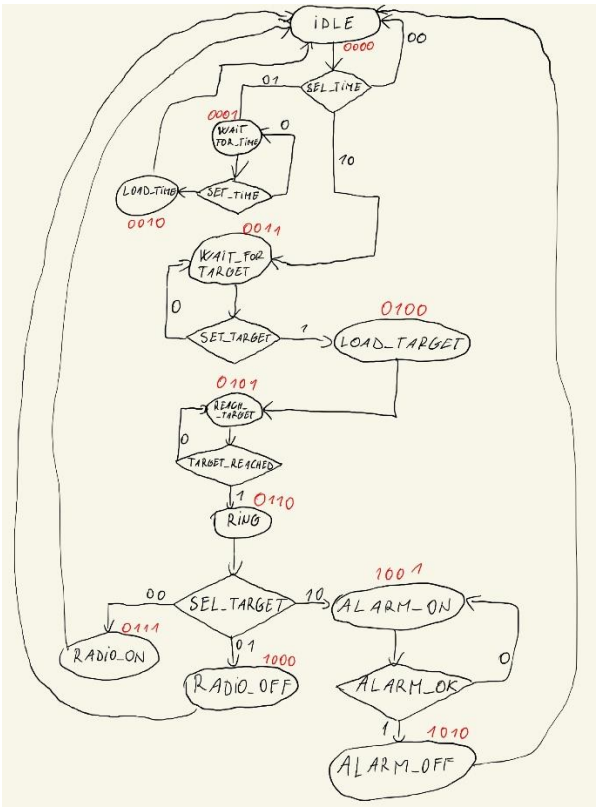


Fig. 4 - Organigrama

3. Componente

În cadrul acestui proiect, două componente esențiale asigură funcționarea corectă a sistemului: contorul pe 17 biți (day_counter) și contorul modular pentru 30 de secunde (alarm_counter). Fiecare dintre ele îndeplinește un rol bine definit în logica generală a ceasului cu alarmă.

Contorul pe 17 biți, denumit day_counter, este responsabil pentru menținerea timpului curent exprimat în secunde. Având în vedere că o zi are exact 86.400 secunde, au fost necesari 17 biți pentru a putea contoriza valori între 0 și 86.399, iar după atingerea limitei, contorul revine la 0, simulând astfel trecerea la o nouă zi. Acest contor este incrementat la fiecare impuls de ceas, în condițiile în care semnalul enable este activ. În plus, modulul include o funcție de setare (set_time) care permite inițializarea contorului cu o valoare specifică, utilă atât în faza de testare, cât și pentru sincronizarea manuală a orei.

```
entity day_counter is
    Port (
        clk     : in  STD_LOGIC;
        rst     : in  STD_LOGIC;
        enable  : in  STD_LOGIC;
        set_time : in std_logic;
        time_in : in STD_LOGIC_VECTOR(16 downto 0);
        count : out STD_LOGIC_VECTOR(16 downto 0)
    );
end day_counter;

architecture Behavioral of day_counter is
    signal count_in : STD_LOGIC_VECTOR(16 downto 0);  -- 17 bits
begin

    process(clk, rst)
    begin
        if rst = '1' then
            count_in <= (others => '0');

        elsif rising_edge(clk) then
            if set_time = '1' then
                count_in <= time_in;
            end if;
            if enable = '1' then
                if count_in = "10101000101111111" then  -- 86399 in binary
                    count_in <= (others => '0');
                else
                    count_in <= count_in + 1;
                end if;
            end if;
        end if;
    end process;

    count <= count_in;
```

A doua componentă importantă este alarm_counter, un contor modular care permite menținerea activă a unei alarme (sonerie sau radio) pentru o durată de 30 de secunde. Acest contor intern este activat atunci când o alarmă este pornită, iar la fiecare impuls de ceas, dacă semnalul enable este activ, contorul crește de la 0 până la 29. La atingerea acestei valori, semnalul de ieșire alarm_ok devine activ, indicând că durata de alarmă s-a încheiat.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity alarm_counter is
    Port (
        clk     : in  STD_LOGIC;
        rst     : in  STD_LOGIC;
        enable  : in  STD_LOGIC;
        alarm_ok : out std_logic
    );
end alarm_counter;

architecture Behavioral of alarm_counter is
    signal count_in : STD_LOGIC_VECTOR(4 downto 0);
begin

    process(clk, rst)
    begin
        if rst = '1' then
            count_in <= (others => '0');
        elsif rising_edge(clk) then
            if enable = '1' then
                if count_in = "11101" then  -- 29 in bin
                    count_in <= (others => '0');
                else
                    count_in <= count_in + 1;
                end if;
            end if;
        end if;
    end process;

    alarm_ok <= '1' when count_in = "11101" else '0';

end Behavioral;
```

UE este responsabilă pentru gestionarea timpului curent (în secunde), compararea acestuia cu o țintă programată (ora alarmei sau momentul pornirii radioului), și activarea temporizată a acestor acțiuni (ex: alarmă activă 30 secunde).

Componente interne în UE:

1. **day_counter (Contor de 17 biți):**
    - Numără secundele dintr-o zi (0 → 86399).
    - Se resetează automat după 86400 secunde.
    - Suportă setarea directă a valorii (set_time), utilă pentru sincronizarea ceasului.
2. **alarm_counter (Contor modulo 30):**
    - Activează semnalul alarm_ok după 30 secunde de la activarea alarmei.
    - Asigură că alarma rămâne pornită exact 30 secunde.

3. **Compararea orei curente cu ținta (target_time):**
    - Dacă q_time = target_time, se activează semnalul target_reached.
    - Acesta este semnalul care declanșează acțiunile din UC (ex: alarmă sau radio).

PORT-uri:

- clk — Semnal de ceas. Toate operațiile sunt sincronizate cu frontal pozitiv.
- rst — Semnal de reset asincron – resetează contorul și logica internă.
- time_in — Valoare de timp introdusă de utilizator, fie pentru ora curentă, fie pentru țintă.
- enable — Activează incrementarea contorului de timp (day_counter).
- radio_on — Semnal de control primit de la UC – pornește radioul.
- radio_off — Semnal de control de oprire a radioului.
- alarm_on — Semnal de control – pornește alarma.
- alarm_off — Semnal de control – oprește alarma.
- set_time — Permite încărcarea valorii time_in ca timp curent (în day_counter).
- set_target — Permite încărcarea valorii time_in ca moment țintă.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ue is
    Port ( clk : in STD_LOGIC;
           rst : in STD_LOGIC;
           time_in : in std_logic_vector(16 downto 0); -- 131071 = 2^17 - 1, 86400 seconds in a day, we need 17 bits
           enable : in std_logic;
           radio_on : in STD_LOGIC;
           radio_off : in STD_LOGIC;
           alarm_on : in STD_LOGIC;
           alarm_off : in STD_LOGIC;
           set_time : in STD_LOGIC;
           set_target : in STD_LOGIC;
           target_reached : out STD_LOGIC;
           alarm_ok : out STD_LOGIC;
           target_time_out : out std_logic_vector(16 downto 0);
           q_time_out : out std_logic_vector(16 downto 0)
           );
end ue;

architecture Behavioral of ue is

component day_counter is
    Port (
        clk     : in  STD_LOGIC;
        rst     : in  STD_LOGIC;
        enable  : in  STD_LOGIC;
        set_time : in std_logic;
        time_in : in STD_LOGIC_VECTOR(16 downto 0);
        count : out STD_LOGIC_VECTOR(16 downto 0)
    );
end component;

component alarm_counter is
    Port (
```

```
component alarm_counter is
    Port (
        clk     : in  STD_LOGIC;
        rst     : in  STD_LOGIC;
        enable  : in  STD_LOGIC;
        alarm_ok : out std_logic
    );
end component;

signal q_time : std_logic_vector(16 downto 0);
signal target_time : std_logic_vector(16 downto 0);

begin

day_cnt: day_counter
    port map (
        clk => clk,
        rst => rst,
        enable => enable,
        set_time => set_time,
        time_in => time_in,
        count => q_time
    );

alarm_cnt: alarm_counter
     port map (
       clk => clk,
       rst => rst,
       enable => alarm_on,
       alarm_ok => alarm_ok
     );

    process(clk, rst)
     begin
         if rst = '1' then
```

```
    process(clk, rst)
     begin
         if rst = '1' then
             target_time <= (others => '0');
             target_reached <= '0';
         elsif q_time = target_time then
             target_reached <= '1';
         else
             target_reached <= '0';
         end if;
         if rising_edge(clk) then
             if set_target = '1' then
                 target_time <= time_in;
                 target_reached <= '0';
             end if;
         end if;
     end process;

    target_time_out <= target_time;
    q_time_out <= q_time;
end Behavioral;
```

UC gestionează logica de trecere între stări, pe baza comenzilor utilizatorului și a semnalelor din UE. Decide când să seteze ora, când să seteze o țintă, ce acțiune să facă la momentul țintă (alarmă/radio) și când să închidă alarma.

Automatul de stare din UC:

- **idle** – așteaptă comenzi
- **wait_4_time → load_time** – permite setarea timpului curent
- **wait_4_target → load_target** – permite setarea momentului țintă (pentru radio/alarmă)
- **reach_target** – așteaptă ca ora curentă să ajungă la țintă
- **ring** – stare de decizie, alege între alarmă sau radio
- **radio_on / radio_off** – activează/dezactivează radioul

- **alarm_on / alarm_off** – activează/dezactivează alarma, în funcție de semnalul alarm_ok (după 30 secunde)

Semnale de intrare importante:

- sel_time – selectează ce operație se face: idle, setare oră, setare țintă
- sel_target – selectează tipul de țintă: radio on, radio off, alarmă
- set_time, set_target – comenzi pentru încărcare
- target_reached, alarm_ok – semnale de la UE

Semnale de ieșire către UE:

- set_time_ue, set_target_ue – comenzi de scriere în contor
- radio_on_ue, radio_off_ue, alarm_on_ue, alarm_off_ue – comenzi care pornesc/opresc radio/sonerie, enable_ue – permite contorizarea timpului în day_counter

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity uc is
    Port ( clk : in STD_LOGIC;
           rst : in STD_LOGIC;
           enable : in STD_LOGIC;
           sel_time : in STD_LOGIC_VECTOR (1 downto 0);
           sel_target : in STD_LOGIC_VECTOR (1 downto 0);
           set_time : in STD_LOGIC;
           set_target : in STD_LOGIC;
           target_reached : in STD_LOGIC;
           alarm_ok : in std_logic;
           enable_ue : out std_logic;
           set_time_ue : out STD_LOGIC;
           set_target_ue : out STD_LOGIC;
           radio_on_ue : out STD_LOGIC;
           radio_off_ue : out STD_LOGIC;
           alarm_on_ue : out STD_LOGIC;
           alarm_off_ue : out std_logic
           );
end uc;

architecture Behavioral of uc is
type state_type is (idle, wait_4_time, load_time, wait_4_target, load_target, reach_target, ring, radio_on, radio_off, alarm_on, alarm_off);
signal state : state_type;
begin

    process(clk, rst)
    begin
        if rst = '1' then
            state <= idle;
        elsif rising_edge(clk) then
            case state is
            when idle =>
```

```
                if sel_time = "00" then
                    state <= idle;
                elsif sel_time = "01" then
                    state <= wait_4_time;
                elsif sel_time = "10" then
                    state <= wait_4_target;
                end if;
            when wait_4_time =>
                if set_time = '0' then
                    state <= wait_4_time;
                else
                    state <= load_time;
                end if;
            when load_time =>
                state <= idle;
            when wait_4_target =>
                if set_target = '0' then
                    state <= wait_4_target;
                else
                    state <= load_target;
                end if;
            when load_target =>
                state <= reach_target;
            when reach_target =>
                if target_reached = '0' then
                    state <= reach_target;
                else
                    report "state is ring";
                    state <= ring;
                end if;
            when ring =>
                if sel_target = "00" then
                    state <= radio_on;
                elsif sel_target = "01" then
                    state <= radio_off;
                elsif sel_target = "10" then
```

```
            when ring =>
                if sel_target = "00" then
                    state <= radio_on;
                elsif sel_target = "01" then
                    state <= radio_off;
                elsif sel_target = "10" then
                    state <= alarm_on;
                end if;
            when radio_on =>
                state <= idle;
            when radio_off =>
                state <= idle;
            when alarm_on =>
                if alarm_ok = '0' then
                    state <= alarm_on;
                else
                    state <= alarm_off;
                end if;
            when alarm_off =>
                state <= idle;
            end case;
        end if;
    end process;

    enable_ue <= enable;
    set_time_ue <= '1' when state = load_time else '0';
    set_target_ue <= '1' when state = load_target else '0';
    radio_on_ue <= '1' when state = radio_on else '0';
    radio_off_ue <= '1' when state = radio_off else '0';
    alarm_on_ue <= '1' when state = alarm_on else '0';
    alarm_off_ue <= '1' when state = alarm_off else '0';
end Behavioral;
```

4. COD VHDL

Fișierul top.vhdl reprezintă componenta principală a proiectului și are rolul de a interconecta cele două blocuri fundamentale: **UC** (Unitatea de Control) și **UE** (Unitatea de Execuție). Acest modul gestionează schimbul de semnale dintre cele două componente și expune la exterior intrările/ieșirile necesare utilizatorului și simulatorului.

Modulul top.vhdl conține două instanțieri:

- **uc_comp**, care este instanța unității de control (uc), responsabilă pentru logica de decizie și comutarea între stări (setare oră, setare alarmă, pornire/oprire alarmă/radio etc.).
- **ue_comp**, care este instanța unității de execuție (ue), unde se află contorul principal (day_counter), logica de comparare a timpului țintă și contorul modular de 30 secunde pentru alarmă.

Semnalele interne generate de uc (precum set_time_ue, alarm_on_ue, radio_on_ue etc.) sunt transmise către ue, care le interpretează și generează semnale de stare (ex: target_reached, alarm_ok).

Aceste semnale sunt apoi transmise înapoi către uc, care poate decide tranziția în alte stări și controlul activ/dezactiv al ieșirilor vizibile la exterior.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity top is
    Port ( clk : in STD_LOGIC;
           rst : in STD_LOGIC;
           enable : in STD_LOGIC;
           set_time : in STD_LOGIC;
           set_target : in STD_LOGIC;
           sel_time : in STD_LOGIC_VECTOR (1 downto 0);
           sel_target : in STD_LOGIC_VECTOR (1 downto 0);
           time_in : in STD_LOGIC_VECTOR (16 downto 0);
           target_time : out STD_LOGIC_VECTOR (16 downto 0);
           q_time : out STD_LOGIC_VECTOR (16 downto 0);
           target_reached_out : out std_logic;
           alarm_on : out STD_LOGIC;
           alarm_off : out STD_LOGIC;
           radio_on : out STD_LOGIC;
           radio_off : out STD_LOGIC);
end top;

architecture Behavioral of top is
component ue is
    Port ( clk : in STD_LOGIC;
           rst : in STD_LOGIC;
           time_in : in std_logic_vector(16 downto 0); -- 131071 = 2^17 - 1, 86400 seconds
           enable : in std_logic;
           radio_on : in STD_LOGIC;
           radio_off : in STD_LOGIC;
           alarm_on : in STD_LOGIC;
           alarm_off : in STD_LOGIC;
           set_time : in STD_LOGIC;
           set_target : in STD_LOGIC;
           target_reached : out STD_LOGIC;
           alarm_ok : out STD_LOGIC;
           target_time_out : out std_logic_vector(16 downto 0);
```

```
component uc is
    Port ( clk : in STD_LOGIC;
           rst : in STD_LOGIC;
           enable : in STD_LOGIC;
           sel_time : in STD_LOGIC_VECTOR (1 downto 0);
           sel_target : in STD_LOGIC_VECTOR (1 downto 0);
           set_time : in STD_LOGIC;
           set_target : in STD_LOGIC;
           target_reached : in STD_LOGIC;
           alarm_ok : in std_logic;
           enable_ue : out std_logic;
           set_time_ue : out STD_LOGIC;
           set_target_ue : out STD_LOGIC;
           radio_on_ue : out STD_LOGIC;
           radio_off_ue : out STD_LOGIC;
           alarm_on_ue : out STD_LOGIC;
           alarm_off_ue : out std_logic
           );
end component;

signal q_time_in : STD_LOGIC_VECTOR (16 downto 0);
signal target_reached : STD_LOGIC;
signal alarm_ok : std_logic;
signal enable_ue : std_logic;
signal set_time_ue : STD_LOGIC;
signal set_target_ue : STD_LOGIC;
signal radio_on_ue : STD_LOGIC;
signal radio_off_ue : STD_LOGIC;
signal alarm_on_ue : STD_LOGIC;
signal alarm_off_ue : std_logic;
```

```
        port map (
            clk           => clk,
            rst           => rst,
            enable        => enable,
            sel_time      => sel_time,
            sel_target    => sel_target,
            set_time      => set_time,
            set_target    => set_target,
            target_reached => target_reached,
            alarm_ok      => alarm_ok,
            enable_ue     => enable_ue,
            set_time_ue   => set_time_ue,
            set_target_ue => set_target_ue,
            radio_on_ue   => radio_on_ue,
            radio_off_ue  => radio_off_ue,
            alarm_on_ue   => alarm_on_ue,
            alarm_off_ue  => alarm_off_ue
        );

ue_comp: ue
        port map (
            clk           => clk,
            rst           => rst,
            time_in       => time_in,
            enable        => enable_ue,
            radio_on      => radio_on_ue,
            radio_off     => radio_off_ue,
            alarm_on      => alarm_on_ue,
            alarm_off     => alarm_off_ue,
            set_time      => set_time_ue,
            set_target    => set_target_ue,
            target_reached => target_reached,
            alarm_ok      => alarm_ok,
            target_time_out => target_time,
            q_time_out => q_time
        );
```

```
    alarm_on <= alarm_on_ue;
    alarm_off <= alarm_off_ue;
    radio_on <= radio_on_ue;
    radio_off <= radio_off_ue;
    target_reached_out <= target_reached;

end Behavioral;
```

5. Utilizare și semnificația notațiilor

PORT-uri de intrare:

- **clk – semnalul de ceas al sistemului; controlează sincronizarea tuturor componentelor.**
- **rst – semnalul de reset global; aduce sistemul într-o stare inițială cunoscută.**
- **enable – semnal de activare generală a execuției ceasului (activează contorul de zi).**
- **set_time – semnal ce permite încărcarea unei valori inițiale în contorul principal (setează ora curentă).**
- **set_target – semnal care permite încărcarea unui moment țintă în unitatea de execuție (ora la care pornește alarma sau radioul).**
- **sel_time – selector pe 2 biți care indică ce operațiune legată de timp se dorește:**
  - **"00" – nicio acțiune.**
  - **"01" – intră în modul de setare a timpului curent.**
  - **"10" – intră în modul de setare a timpului țintă (pentru alarmă/radio).**
- **sel_target – selector pe 2 biți care indică acțiunea de făcut la momentul țintă:**
  - **"00" – se pornește radioul.**
  - **"01" – se oprește radioul.**
  - **"10" – se pornește alarma.**
- **time_in – valoarea pe 17 biți ce reprezintă fie ora curentă, fie ora țintă, în funcție de contextul semnalelor set_time sau set_target.**

PORT-uri ieșire:

- **target_time** – arată valoarea stocată ca moment țintă, pentru verificări sau afișare.
- **q_time** – ieșirea contorului de timp (timpul curent, în secunde).
- **target_reached_out** – semnal care devine 1 când timpul curent ajunge la timpul țintă.
- **alarm_on** – semnal care indică faptul că alarma a fost pornită.
- **alarm_off** – semnal care indică faptul că alarma s-a oprit (automat, după 30 de secunde).
- **radio_on** – semnal care comandă pornirea radioului.
- **radio_off** – semnal care comandă oprirea radioului.

MANUAL DE UTILIZARE

1. Descriere generala

Acest proiect VHDL reprezintă un **ceas digital programabil**, care contorizează secundele unei zile folosind un **contor de 17 biți** (până la 86.399 secunde, echivalentul unei zile complete: 24h x 60min x 60sec).
Sistemul permite utilizatorului să:

- seteze ora curentă (în secunde de la miezul nopții),
- programeze o acțiune automată la o anumită oră (ex: radio ON, radio OFF, alarmă),
- activeze o alarmă pentru o durată fixă de 30 secunde.

2. Functionalitate

- Contorizare precisă a timpului în secunde pe o zi (modulo 86400).
- Setare oră curentă (set_time) și oră de declanșare (set_target).
- Alegerea acțiunii asociate momentului țintă:

- Radio ON (sel_target = 00)
- Radio OFF (sel_target = 01)
- Alarmă timp de 30 secunde (sel_target = 10)

 Comportament automat bazat pe comparația dintre timpul curent și cel programat.
 Automat de stare (UC) care controlează logica generală.
 Contor de alarmă ce menține alarma activă timp de exact 30 secunde

3. Crearea și încărcarea proiectului în Vivado

1. Crearea proiectului

1. Deschide **Vivado**.
2. Selectează **Create New Project**.
3. Alege un nume (ex: ceas_programabil) și o locație.
4. Selectează opțiunea **RTL Project** și debifează „Add sources” temporar.
5. Finalizează proiectul fără a selecta nicio placă (dacă vrei doar simulare).

2. Adăugarea fișierelor sursă

1. Click dreapta pe „**Design Sources**” > Add Sources > Add or Create Design Sources.
2. Adaugă fișierele:
   - top.vhdl (modul principal)

- uc.vhdl (unitatea de control)
- ue.vhdl (unitatea de execuție)
- day_counter.vhdl (contor de 17 biți)
- alarm_counter.vhdl (contor de 30 secunde)

3. Adăugarea testbench-ului

1. Click dreapta pe „**Simulation Sources**” > Add Sources > Add or Create Simulation Sources.
2. Adaugă fișierul tb_top.vhdl sau echivalentul tău.
3. Asigură-te că fișierul are architecture Behavioral of tb_top is și conține stimuli corespunzători pentru test.

4. Rularea simulării

1. În Vivado, selectează Run Simulation > Run Behavioral Simulation.
2. După încărcare:
   - Vizualizează semnalele dorite (q_time, target_time, alarm_on, radio_on, etc.).
   - Rulează simularea (Run for X ns, Run All).
3. Poți analiza:
   - Momentul în care se atinge ora programată (target_reached_out = 1).
   - Activarea semnalelor radio_on, alarm_on, alarm_off etc.
   - Timpul intern (q_time) și timpul țintă (target_time).

6. Justificarea soluției alese

Am ales să folosesc un singur contor pe 17 biți pentru a reprezenta timpul în secunde, deoarece simplifică mult proiectul. Față de varianta cu 3 contoare (secunde, minute, ore), această soluție reduce numărul de semnale și comparări necesare. Compararea timpului actual cu ora unei alarme se face direct, fără conversii.

Am separat clar unitatea de control (UC) de unitatea de execuție (UE), ceea ce oferă structură modulară, ușor de extins și testat. Contorul de alarmă menține o alarmă activă fix 30 de secunde, ceea ce ajută la controlul precis al duratei.

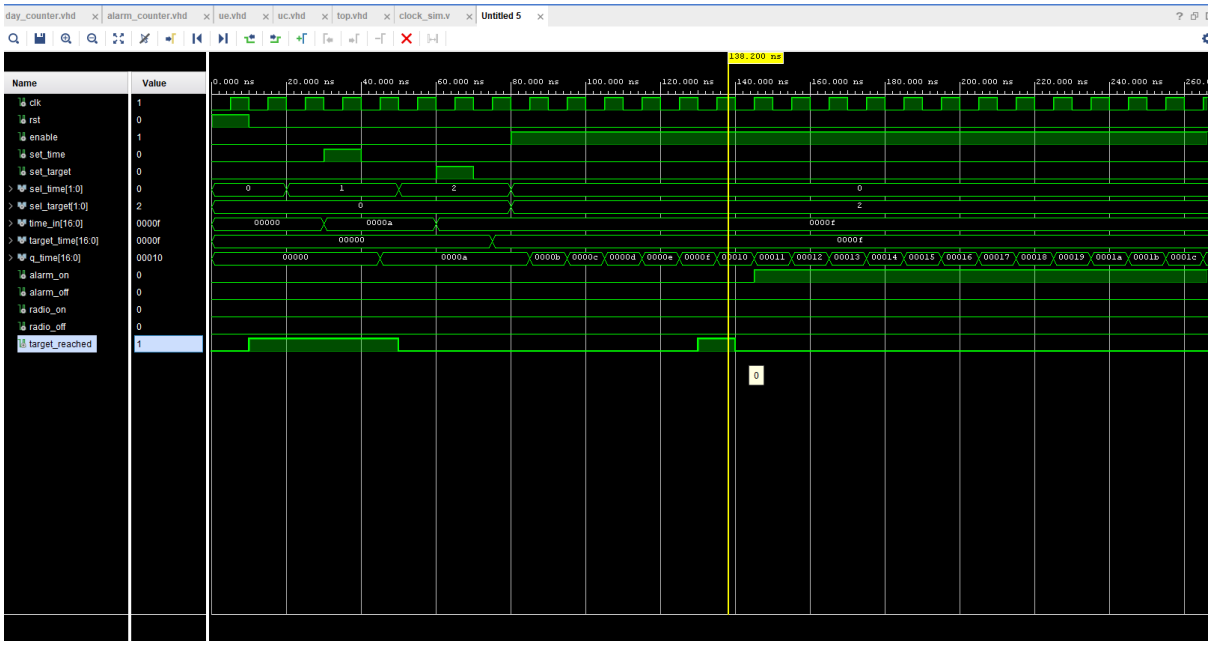
Avantaje:

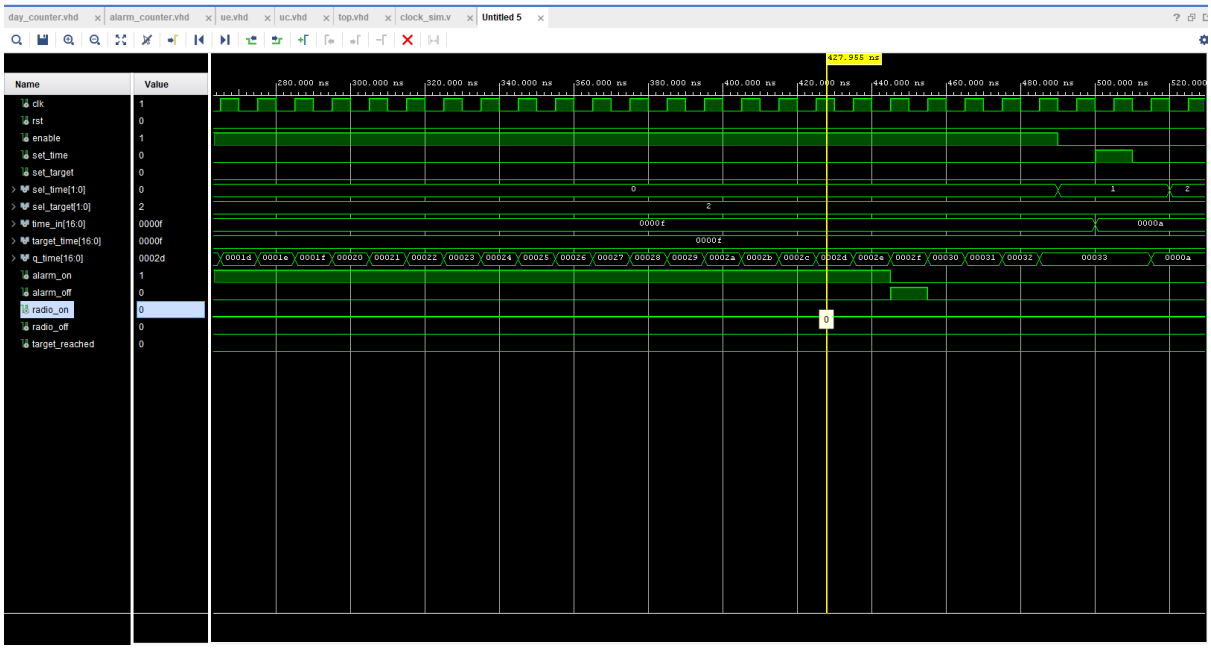
- **Arhitectură simplă și clară;**
- **Comparare directă a orei;**
- **Ușor de testat și extins.**

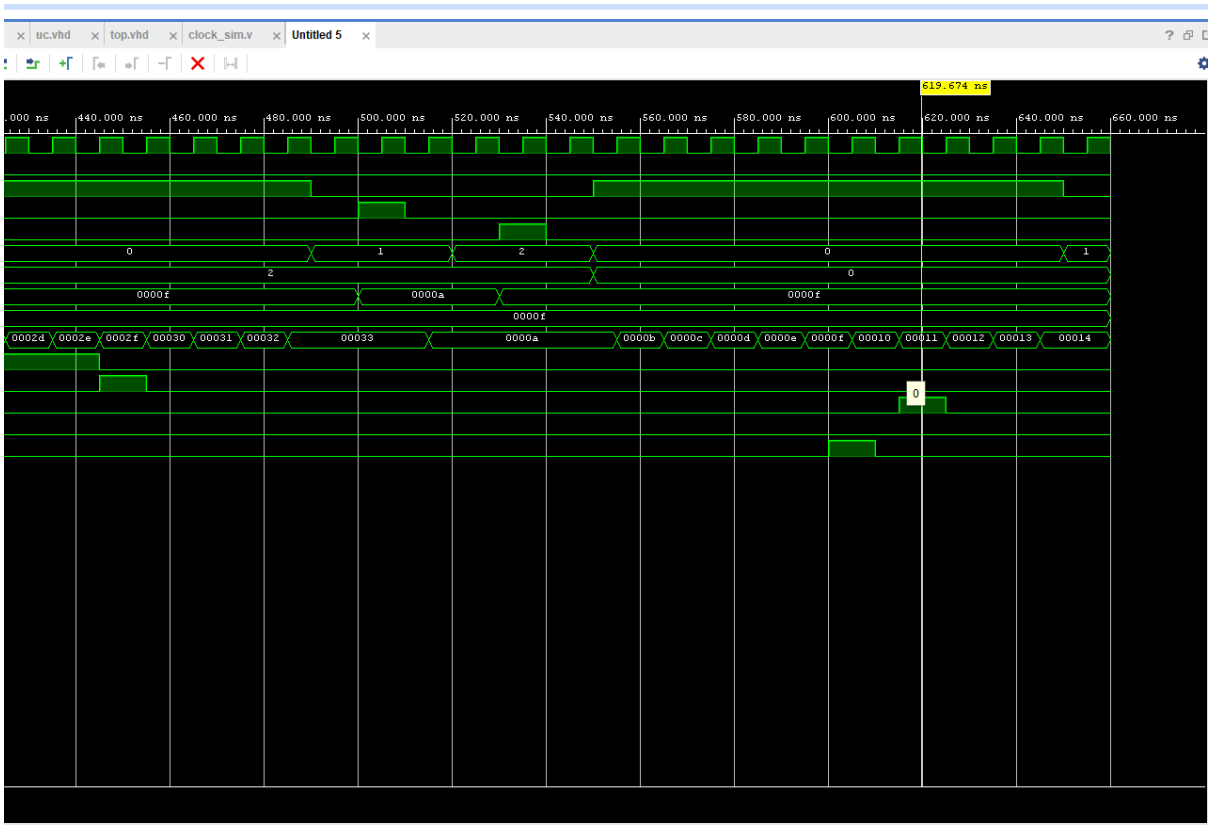
Dezavantaje:

- **Timpul este exprimat doar în secunde, deci trebuie convertit în ore/minute pentru afișare.**

7. Rezultate


day_counter.vhd
alarm_counter.vhd
ue.vhd
uc.vhd
top.vhd
clock_sim.v
Untitled 5
Name
Value
clk
rst
enable
set_time
set_target
sel_time[1:0]
sel_target[1:0]
time_in[16:0]
target_time[16:0]
q_time[16:0]
alarm_on
alarm_off
radio_on
radio_off
target_reached


day_counter.vhd
alarm_counter.vhd
ue.vhd
uc.vhd
top.vhd
clock_sim.v
Untitled 5
Name
Value
clk
rst
enable
set_time
set_target
sel_time[1:0]
sel_target[1:0]
time_in[16:0]
target_time[16:0]
q_time[16:0]
alarm_on
alarm_off
radio_on
radio_off
target_reached



8. Oportunități de dezvoltare

Proiectul actual poate fi îmbunătățit în mai multe moduri utile și realiste.

- prima idee ar fi implementarea unui afișaj digital (de tip 7 segmente sau LCD), care să arate ora curentă în format convenabil (HH:MM:SS), calculată pe baza numărătorului de 17 biți.

- **Adăugarea funcției de snooze**: atunci când alarma este activată, utilizatorul poate apăsa un buton pentru a o amâna automat cu 5–10 minute.

- **Control prin Bluetooth**: Se poate adăuga un modul Bluetooth conectat la FPGA, care să permită utilizatorului să seteze ora și alarma de pe telefon, folosind o aplicație simplă. Această funcționalitate ar elimina nevoia interacțiunii directe prin butoane sau semnale simulate și ar face proiectul mai modern și mai practic.